Charts with Matplotlib

🎯 Today's goal

By the end of this hour you will:

- draw **line charts**, **bar charts**, **histograms** and **scatter plots**
- give every chart a title, axis labels and a legend
- change colours and markers
- put two charts side by side with `plt.subplots`
- draw charts straight from a pandas table, and pick the right chart for a question

🤔 Why this matters

Look at twelve monthly temperatures in a table and you see… twelve numbers. Draw them as a line and you **instantly** see summer rise and winter fall. Our eyes are much better at shapes than at columns of digits.

Charts are how data scientists **explore** data (spotting trends, odd values and patterns) and how they **explain** results to everyone else. Tomorrow's lesson, exploring a whole dataset, leans on today's charts. The tool we use is **Matplotlib**, the most widely used charting library in Python. pandas and many other libraries draw their charts with it behind the scenes.

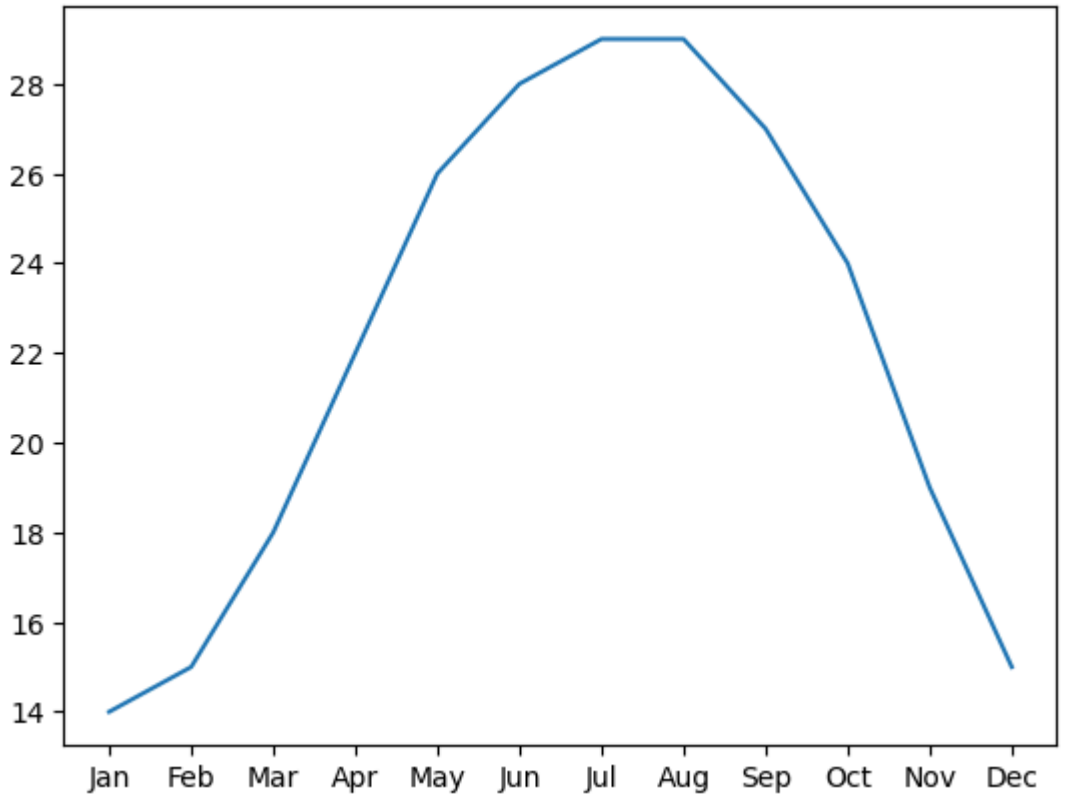
📈 Your first chart: a line

Matplotlib's charting tools live in `matplotlib.pyplot`, which everyone imports with the short name `plt` (like `np` and `pd`). `plt.plot(x, y)` draws a line through the points: the first list goes along the bottom (the **x-axis**), the second goes up the side (the **y-axis**). The temperatures are **made up**:

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
cairo = [14, 15, 18, 22, 26, 28, 29, 29, 27, 24, 19, 15]   # made-up average °C

plt.plot(months, cairo)
plt.show()
```

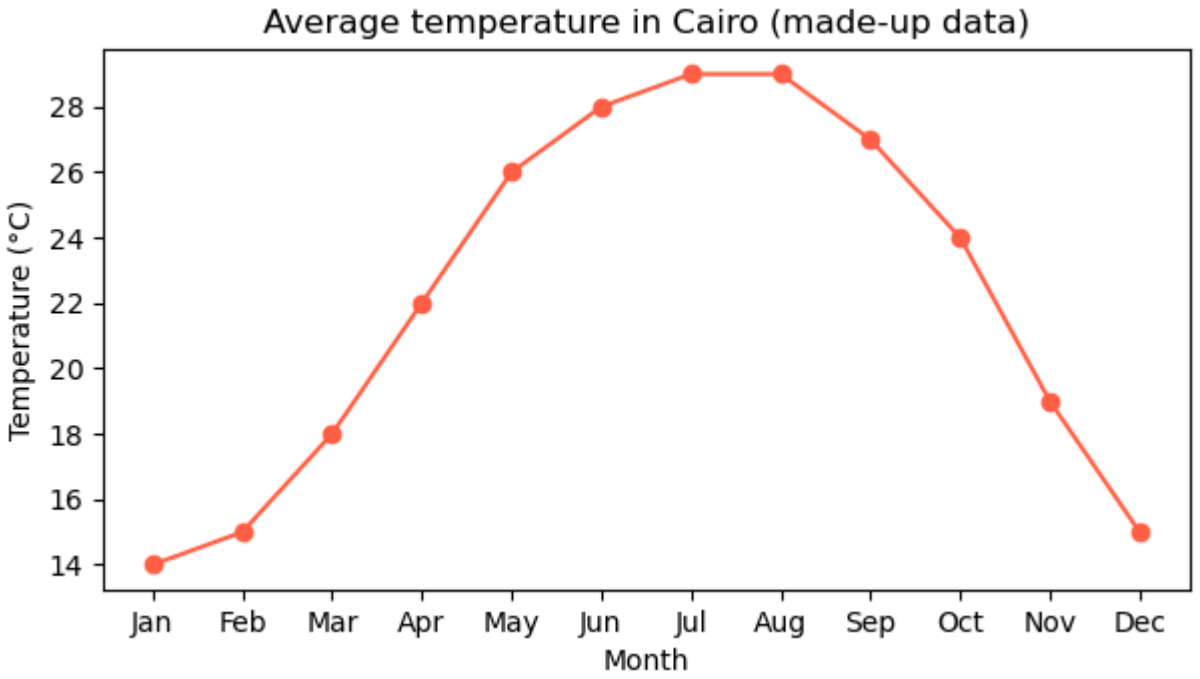


On this page the chart appears by itself under the box. `plt.show()` is the command that displays a chart in normal Python programs, so we keep it at the end of each box; it's a good habit.

🏷️ Titles, labels and style

A chart without labels is a riddle. Always add a **title** and **axis labels** (with units!). You can also choose the size of the chart in inches with `plt.figure(figsize=(width, height))`, a colour, and a **marker**, a small symbol on each data point (`"o"` is a dot, `"s"` a square, `"^"` a triangle):

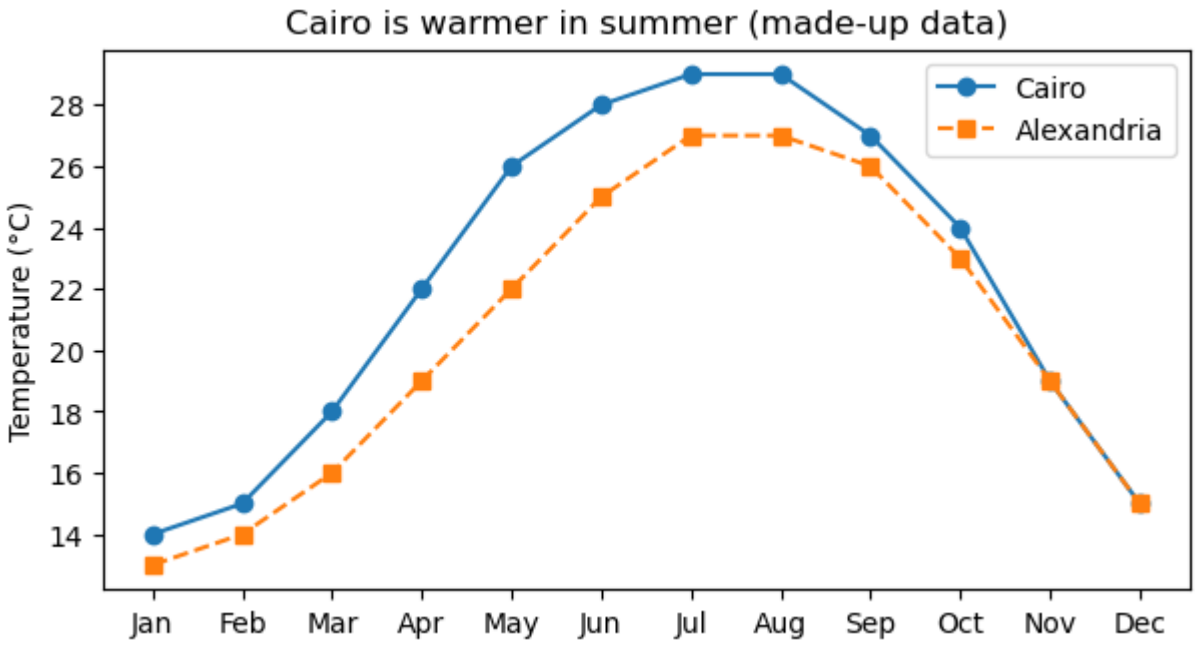
```
plt.figure(figsize=(7, 3.5))
plt.plot(months, cairo, color="tomato", marker="o")
plt.title("Average temperature in Cairo (made-up data)")
plt.xlabel("Month")
plt.ylabel("Temperature (°C)")
plt.show()
```



To compare, call `plt.plot` twice. Give each line a `label`, and `plt.legend()` shows a little key that says which line is which:

```
alexandria = [13, 14, 16, 19, 22, 25, 27, 27, 26, 23, 19, 15]   # made-up

plt.figure(figsize=(7, 3.5))
plt.plot(months, cairo, marker="o", label="Cairo")
plt.plot(months, alexandria, marker="s", linestyle="--", label="Alexandria")
plt.title("Cairo is warmer in summer (made-up data)")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.show()
```

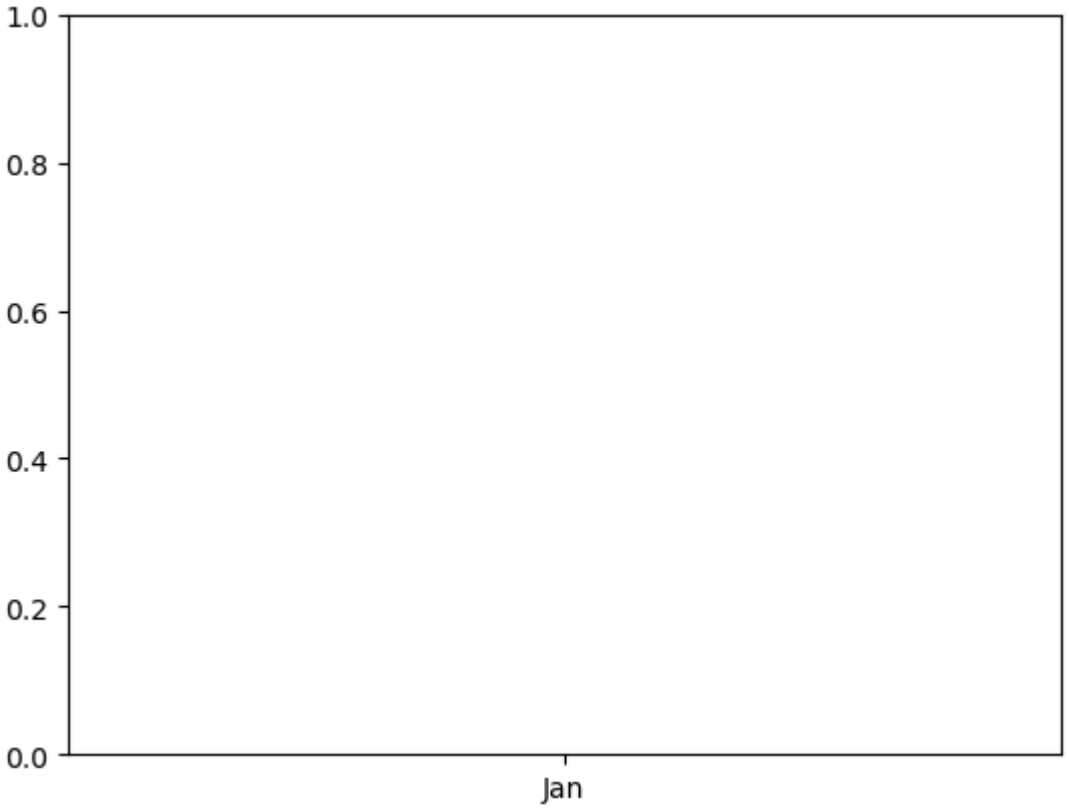


`linestyle="--"` makes a dashed line. A good title says what the reader should notice, not just what the data is.

The two lists must have the **same length**: one y value for each x value. Here one month is missing:

💡 This example has a mistake on purpose. Run it and read the error.

```
short_list = [14, 15, 18, 22, 26, 28, 29, 29, 27, 24, 19]   # only 11 numbers
plt.plot(months, short_list)
```



⚠️ **ValueError**

The kind of value is right but the value itself isn't, like int("hello").

```
Traceback (most recent call last):
  line 2, in <module>
    plt.plot(months, short_list)
    ~~~~~~~~^^^^^^^^^^^^^^^^^^^^
ValueError: x and y must have same first dimension, but have shapes (12,) and
(11,)
```

"x and y must have same first dimension" means "I got 12 x values but only 11 y values". Count your data with `len()` when you see this. (The empty frame is the chart Matplotlib had already started before it gave up.)

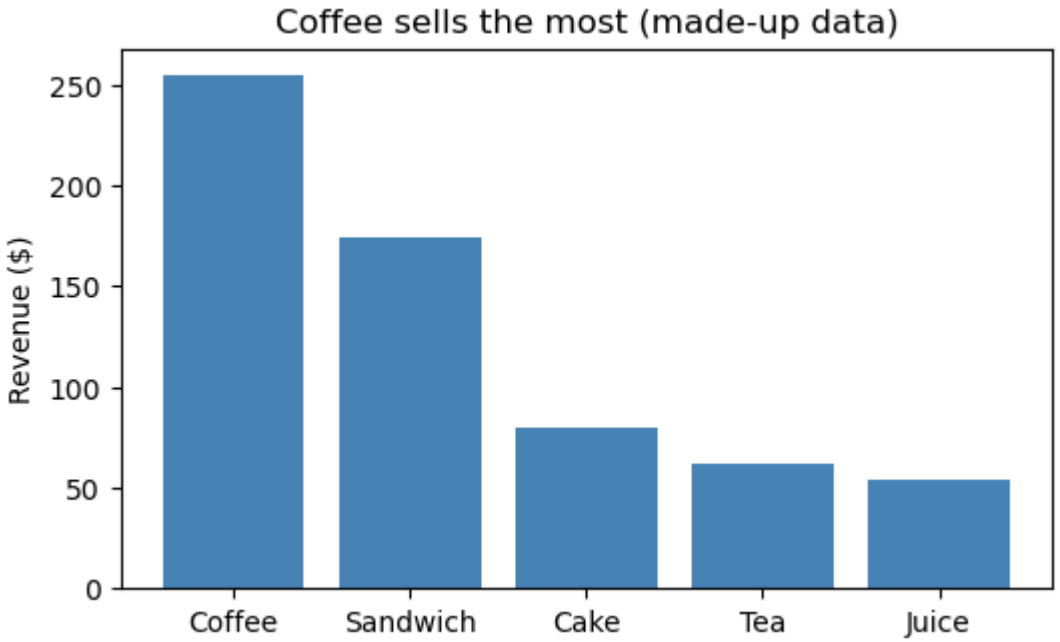
📊 Bar charts: comparing categories

A line suits values that follow each other in time. To compare separate **categories** (products, cities, shops) use bars. `plt.bar` takes the names and the heights. Here are made-up weekly sales of a

café:

```
product_sales = {"Coffee": 255, "Sandwich": 174, "Cake": 80,
                 "Tea": 62, "Juice": 54}
names = list(product_sales.keys())
amounts = list(product_sales.values())

plt.figure(figsize=(6, 3.5))
plt.bar(names, amounts, color="steelblue")
plt.title("Coffee sells the most (made-up data)")
plt.ylabel("Revenue ($)")
plt.show()
```

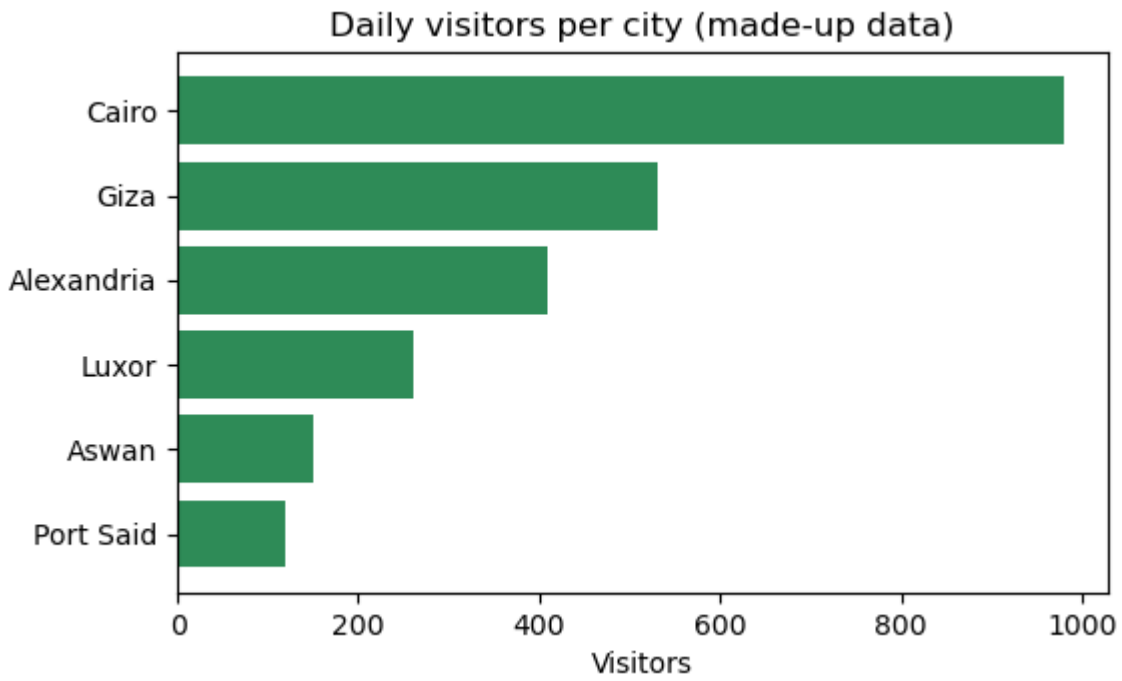


When names are long, or there are many of them, **horizontal** bars are easier to read: `plt.barh`. Sorting the bars makes the ranking jump out:

```
import pandas as pd

city_visitors = pd.Series({
    "Alexandria": 410, "Cairo": 980, "Giza": 530,
    "Luxor": 260, "Aswan": 150, "Port Said": 120,
})                                              # made-up visitors per day
city_visitors = city_visitors.sort_values()   # smallest first = at the bottom

plt.figure(figsize=(6, 3.5))
plt.barh(city_visitors.index, city_visitors.values, color="seagreen")
plt.title("Daily visitors per city (made-up data)")
plt.xlabel("Visitors")
plt.show()
```



A Series works nicely here: `.index` holds the names and `.values` the numbers.

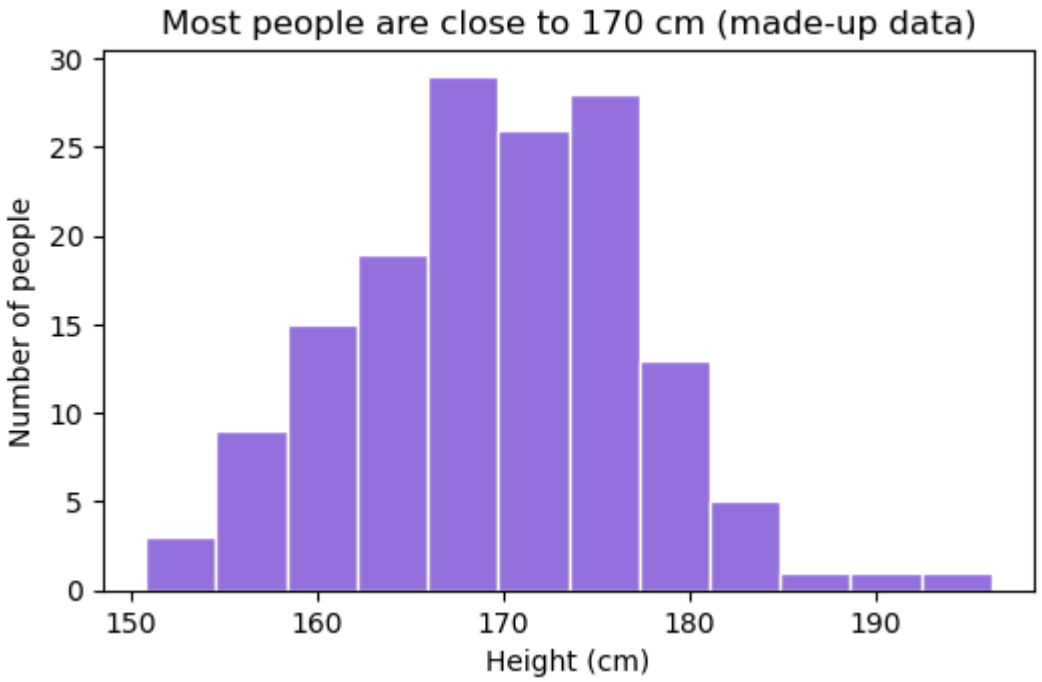
📶 Histograms: how values are spread

A **histogram** answers "which values are common and which are rare?". It cuts the range of values into equal slices called **bins** and draws one bar per bin, as tall as the number of values that fall into it. Let's make 150 made-up adult heights with NumPy's random generator (Day 9). The seed `42` makes the "random" numbers the same every time:

```
import numpy as np

rng = np.random.default_rng(42)
heights = rng.normal(170, 9, 150).round(1)    # 150 made-up heights (cm)

plt.figure(figsize=(6, 3.5))
plt.hist(heights, bins=12, color="mediumpurple", edgecolor="white")
plt.title("Most people are close to 170 cm (made-up data)")
plt.xlabel("Height (cm)")
plt.ylabel("Number of people")
plt.show()
```



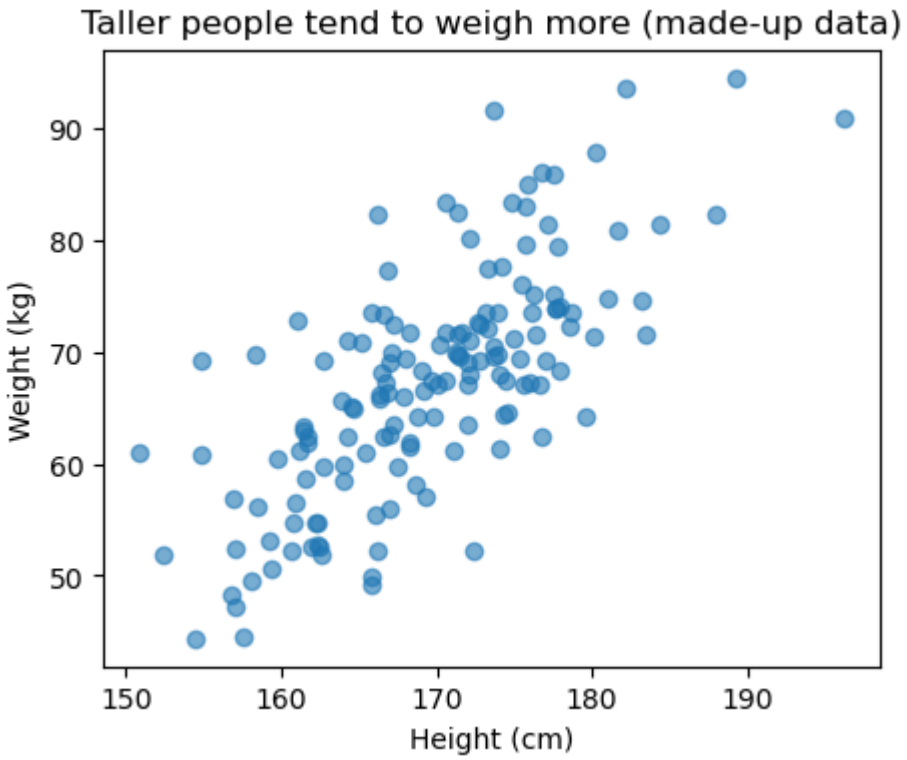
The typical "hill" shape: lots of people in the middle, few very short or very tall ones. Try `bins=4` and `bins=40`: too few bins hide the shape, too many make it spiky. Somewhere between 10 and 30 is usually a good start.

⚫ Scatter plots: do two things go together?

A **scatter plot** draws one dot per person (or shop, or day), placed by two numbers at once. It shows whether two measurements move together. Do taller people weigh more? Each dot below is one made-up person:

```
# Made-up weights that loosely follow the heights
weights = (0.9 * heights - 85 + rng.normal(0, 7, 150)).round(1)

plt.figure(figsize=(5, 4))
plt.scatter(heights, weights, alpha=0.6)
plt.title("Taller people tend to weigh more (made-up data)")
plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")
plt.show()
```



The dots form a cloud that slopes upwards: on average, more height comes with more weight, though not for everyone. `alpha=0.6` makes the dots a little see-through, so you can spot where many overlap. (Tomorrow you'll measure this "going together" with a number.)

🪟 Two charts side by side

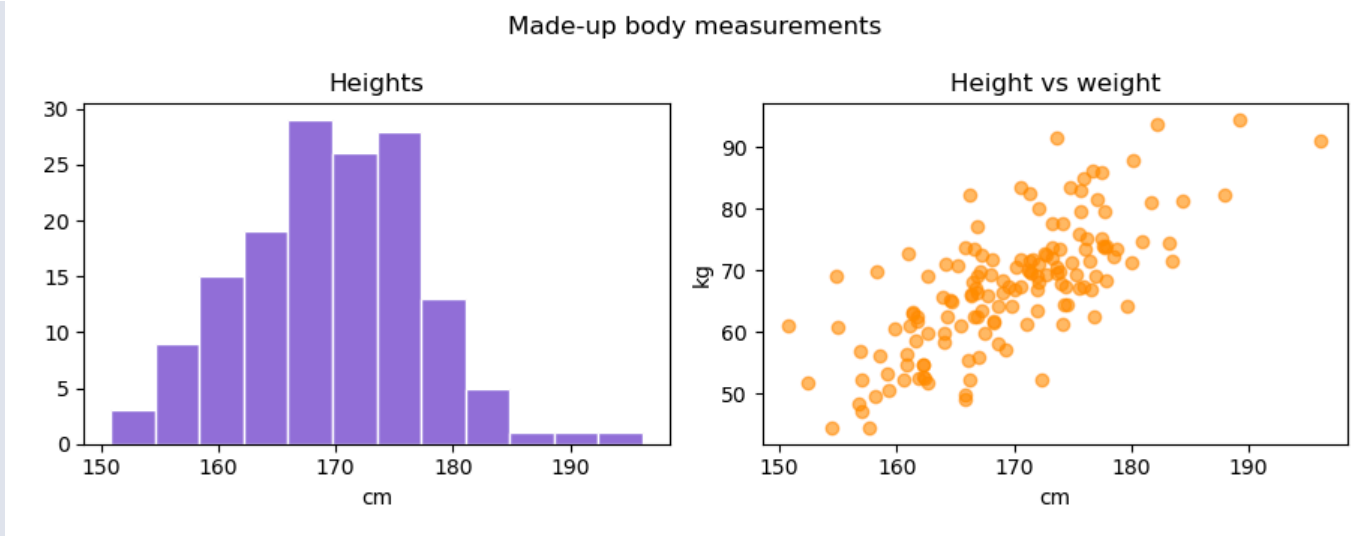
`plt.subplots(1, 2)` makes a figure with **1 row and 2 columns** of charts. It gives back two things: the whole figure (`fig`) and the separate charts, called **axes**. You draw on each one with the same commands, called on the axes instead of on `plt` . Labels are set with `set_title` , `set_xlabel` and `set_ylabel` :

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.5))

left.hist(heights, bins=12, color="mediumpurple", edgecolor="white")
left.set_title("Heights")
left.set_xlabel("cm")

right.scatter(heights, weights, alpha=0.6, color="darkorange")
right.set_title("Height vs weight")
right.set_xlabel("cm")
right.set_ylabel("kg")

fig.suptitle("Made-up body measurements")   # one title over both charts
plt.tight_layout()                           # stop labels from overlapping
plt.show()
```



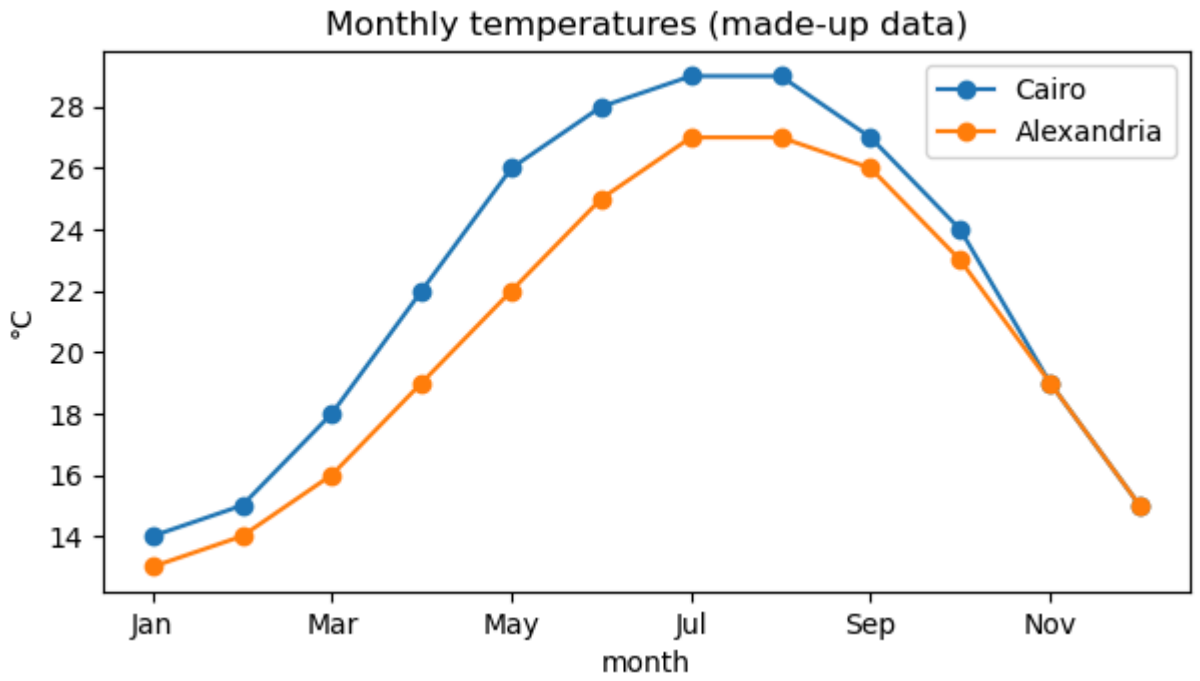
`fig, (left, right) = ...` unpacks the two results into variables in one line, the same way a function can return several values (Day 6).

🐼 Charts straight from pandas

pandas tables and Series have a `.plot()` method that calls Matplotlib for you and even adds labels from the column names. Use `kind=` to choose the chart: `"line"`, `"bar"`, `"barh"`, `"hist"` or `"scatter"`:

```
temps = pd.DataFrame({"month": months, "Cairo": cairo, "Alexandria": alexandria})

temps.plot(x="month", y=["Cairo", "Alexandria"], kind="line", marker="o",
           figsize=(7, 3.5), title="Monthly temperatures (made-up data)")
plt.ylabel("°C")
plt.show()
```



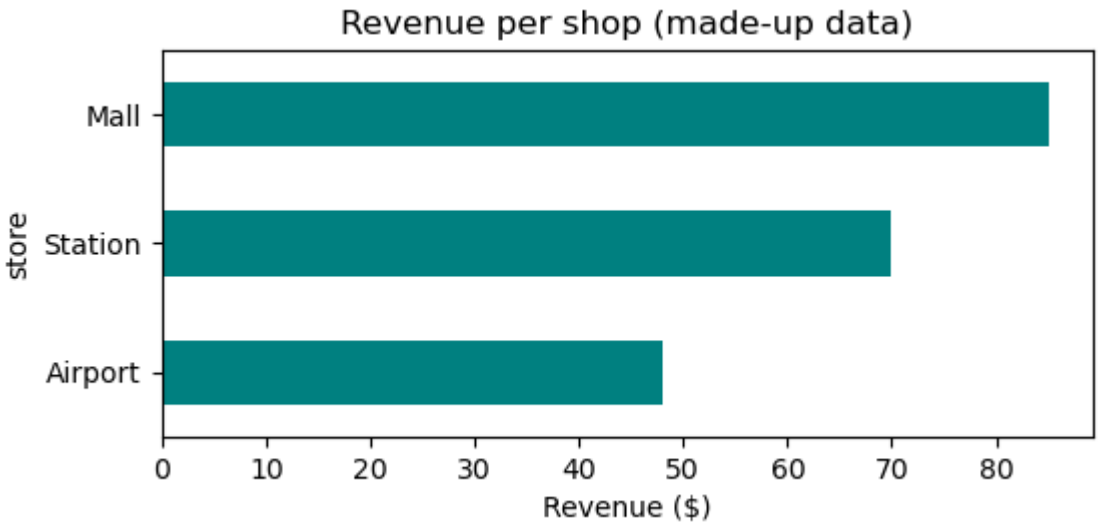
This is especially handy after a `groupby` from Day 11: group, add up, plot. Here are some made-up orders from three café shops:

```
orders = pd.DataFrame({
    "store": ["Airport", "Mall", "Station", "Mall", "Airport",
              "Station", "Mall", "Airport", "Mall", "Station"],
    "revenue": [28.0, 7.5, 36.0, 17.5, 8.0, 31.5, 42.0, 12.0, 18.0, 2.5],
})

store_revenue = orders.groupby("store")["revenue"].sum().sort_values()
print(store_revenue)

store_revenue.plot(kind="barh", color="teal", figsize=(6, 2.5),
                   title="Revenue per shop (made-up data)")
plt.xlabel("Revenue ($)")
plt.show()
```

```
store
Airport    48.0
Station    70.0
Mall       85.0
Name: revenue, dtype: float64
```



🧭 Which chart should I use?

Your question	Chart	Matplotlib
How does something change over time?	line	`plt.plot`
Which category is biggest?	bar (or horizontal bar)	`plt.bar`, `plt.barh`
How are the values spread out? Any odd ones?	histogram	`plt.hist`
Do two measurements go together?	scatter	`plt.scatter`

In normal Python you can save a chart as a picture file, to put in a report or a presentation: write `plt.savefig("sales.png", dpi=150)` just before `plt.show()` (*dpi*, dots per inch, sets the sharpness). On this page you don't need it: every chart already appears under its box, and you can right-click it to save the picture.

⚠️ Common mistakes

- x and y of different lengths → `ValueError: x and y must have same first dimension`.
- No title or axis labels: the reader has to guess what they are looking at. Add units, like "(°C)".
- Using a line chart for separate categories (products, cities). Lines suggest a path in time; use bars.
- On subplots, writing `left.title("...")`. On axes it's `left.set_title("...")` (and `set_xlabel`, `set_ylabel`).
- Calling `plt.legend()` without giving the lines a `label=`: the legend stays empty.

✅ Recap

- `import matplotlib.pyplot as plt`; finish with `plt.show()`.
- Line `plt.plot(x, y)`, bars `plt.bar` / `plt.barh`, histogram `plt.hist(values, bins=…)`, scatter `plt.scatter(x, y)`.
- Labels: `plt.title`, `plt.xlabel`, `plt.ylabel`, `plt.legend()` with `label=` on each line.
- Style: `color=`, `marker=`, `linestyle=`, `alpha=`; size: `plt.figure(figsize=(w, h))`.
- Side by side: `fig, (a, b) = plt.subplots(1, 2)`, then `a.plot(...)`, `a.set_title(...)`.
- pandas: `df.plot(x=..., y=..., kind=...)` and `series.plot(kind="bar")`.
- Save a chart as a picture: `plt.savefig("chart.png")`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Website visitors

Draw a **line chart** of the visitors per day with dots as markers, and give it a title with `plt.title`. Also store the busiest day's number of visitors in `most_visitors` (use `max`).

```
import matplotlib.pyplot as plt

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
visitors = [120, 135, 128, 160, 210, 260, 190]   # made-up

# Draw the chart and create most_visitors
```

Hints:

1. `plt.plot(days, visitors, marker="o")` draws the line with dots.
2. `plt.title("Visitors per day")` adds a title. Then `plt.show()`.
3. `most_visitors = max(visitors)`

2. Fruit sales bar chart

Turn the dictionary into two lists, `fruits` (the names) and `kilos` (the amounts), then draw a **bar chart** with `plt.bar` and a y-axis label with `plt.ylabel`.

```
import matplotlib.pyplot as plt

# Made-up kilos of fruit sold this week
fruit_sales = {"Apples": 42, "Bananas": 58, "Oranges": 35, "Mangoes": 21}

# Create fruits and kilos, then draw the bar chart
```

Hints:

1. `fruits = list(fruit_sales.keys())` and `kilos = list(fruit_sales.values())`
2. `plt.bar(fruits, kilos)`
3. `plt.ylabel("Sold (kg)")`, then `plt.show()`.

3. Exam score histogram

Draw a **histogram** of the 200 made-up exam scores with exactly **8 bins**, and label the x-axis. Also store the average score in `mean_score` (NumPy arrays have `.mean()`).

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
scores = rng.normal(68, 12, 200).clip(0, 100).round()   # 200 made-up scores

# Draw the histogram and create mean_score
```

Hints:

1. `plt.hist(scores, bins=8)`
2. `plt.xlabel("Score")` labels the x-axis.
3. `mean_score = scores.mean()`

4. Study time and scores

Draw a **scatter plot** with study hours along the bottom and scores up the side. Give it a title and both axis labels. Then answer the question in a variable: if higher scores go with more study hours, set `goes_up = True`, otherwise `False`.

```
import matplotlib.pyplot as plt

study_hours = [1, 2, 2, 3, 4, 4, 5, 6, 7, 8]           # made-up
scores = [52, 55, 60, 58, 66, 70, 72, 75, 83, 88]       # made-up

# Draw the scatter plot, then set goes_up
```

Hints:

1. `plt.scatter(study_hours, scores)`: x first, y second.
2. Add `plt.title(...)`, `plt.xlabel(...)` and `plt.ylabel(...)`.
3. Look at the dots: do they climb from left to right? Then `goes_up = True`.

5. Challenge: a two-panel report

Make one figure with **1 row and 2 charts** using `plt.subplots`:

- First calculate the total revenue per store with `groupby` (Day 11) and store it in `store_totals`.
- **Left:** a bar chart of `store_totals`, with a title (`set_title`).
- **Right:** a histogram of the order revenues, with a title.

```
import pandas as pd
import matplotlib.pyplot as plt

orders = pd.DataFrame({
    "store": ["Airport", "Mall", "Station", "Mall", "Airport",
              "Station", "Mall", "Airport", "Mall", "Station"],
    "revenue": [28.0, 7.5, 36.0, 17.5, 8.0, 31.5, 42.0, 12.0, 18.0, 2.5],
})   # made-up

# 1. store_totals
# 2. fig, (left, right) = plt.subplots(...)
# 3. left: bar chart, right: histogram
```

Hints:

1. `store_totals = orders.groupby("store")["revenue"].sum()`
2. `fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.5))`
3. `left.bar(store_totals.index, store_totals.values)` and `right.hist(orders["revenue"], bins=5)`; titles with `left.set_title("...")`.

➡️ Tomorrow

Exploratory data analysis (EDA): you now have all the tools: loading, filtering, grouping, cleaning and charts. Tomorrow you'll put them together into a routine for getting to know a brand-new dataset: ask questions, check for gaps, look at distributions and relationships, find outliers, and write down what you discovered.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Website visitors

```
import matplotlib.pyplot as plt

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
visitors = [120, 135, 128, 160, 210, 260, 190]   # made-up

plt.plot(days, visitors, marker="o")
plt.title("Visitors per day (made-up data)")
plt.show()

most_visitors = max(visitors)
print("Busiest day:", most_visitors, "visitors")
```

2. Fruit sales bar chart

```
import matplotlib.pyplot as plt

# Made-up kilos of fruit sold this week
fruit_sales = {"Apples": 42, "Bananas": 58, "Oranges": 35, "Mangoes": 21}

fruits = list(fruit_sales.keys())
kilos = list(fruit_sales.values())

plt.bar(fruits, kilos, color="orange")
plt.title("Fruit sold this week (made-up data)")
plt.ylabel("Sold (kg)")
plt.show()
```

3. Exam score histogram

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
scores = rng.normal(68, 12, 200).clip(0, 100).round()   # 200 made-up scores

plt.hist(scores, bins=8, edgecolor="white")
plt.title("Exam scores (made-up data)")
plt.xlabel("Score")
plt.ylabel("Number of students")
plt.show()

mean_score = scores.mean()
print(f"Average score: {mean_score:.1f}")
```

4. Study time and scores

```
import matplotlib.pyplot as plt

study_hours = [1, 2, 2, 3, 4, 4, 5, 6, 7, 8]           # made-up
scores = [52, 55, 60, 58, 66, 70, 72, 75, 83, 88]       # made-up

plt.scatter(study_hours, scores)
plt.title("More study, higher scores (made-up data)")
plt.xlabel("Study hours")
plt.ylabel("Score")
plt.show()

goes_up = True
```

5. Challenge: a two-panel report

```
import pandas as pd
import matplotlib.pyplot as plt

orders = pd.DataFrame({
    "store": ["Airport", "Mall", "Station", "Mall", "Airport",
              "Station", "Mall", "Airport", "Mall", "Station"],
    "revenue": [28.0, 7.5, 36.0, 17.5, 8.0, 31.5, 42.0, 12.0, 18.0, 2.5],
})   # made-up

store_totals = orders.groupby("store")["revenue"].sum()

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3.5))
left.bar(store_totals.index, store_totals.values, color="teal")
left.set_title("Revenue per store")
right.hist(orders["revenue"], bins=5, edgecolor="white")
right.set_title("Size of single orders")
plt.tight_layout()
plt.show()
```